

Documentação da Arquitetura SOA — 2F-AGRO / Space Connect

Disciplina: SOA — Service-Oriented Architecture · FIAP 3ESPZ · GS 2026.1 **Repositório:** <https://github.com/GS-SPACE-CONNECT/2f-agro-soa>

Integrantes

Nome	RM
Lucca Borges	554608
Ruan Melo	557599
Rodrigo Jimenez	558148
João Victor Franco	556790
Bruno Leão	555563

1. Descrição da solução, problema e objetivos

Problema

No agronegócio, decisões de campo (irrigação, proteção contra geada, manejo de pragas) dependem de dados de **clima/satélite** e de **cadastros oficiais do governo** (EMATER/MAPA/CAR). Esses sistemas são heterogêneos: o app moderno fala **REST/JSON**, enquanto o sistema legado do governo expõe **SOAP/XML**. Integrá-los exige interoperabilidade real.

Solução

Uma solução de **serviços distribuídos** em **Java + Spring Boot** que: - expõe uma **API REST** para o CRUD da entidade `Propriedade`; - simula o **sistema legado do governo** com um **Web Service SOAP** (cadastro rural); - **integra** os dois e enriquece a propriedade com **dados espaciais externos** (NASA POWER), gerando alertas agroclimáticos.

Objetivos

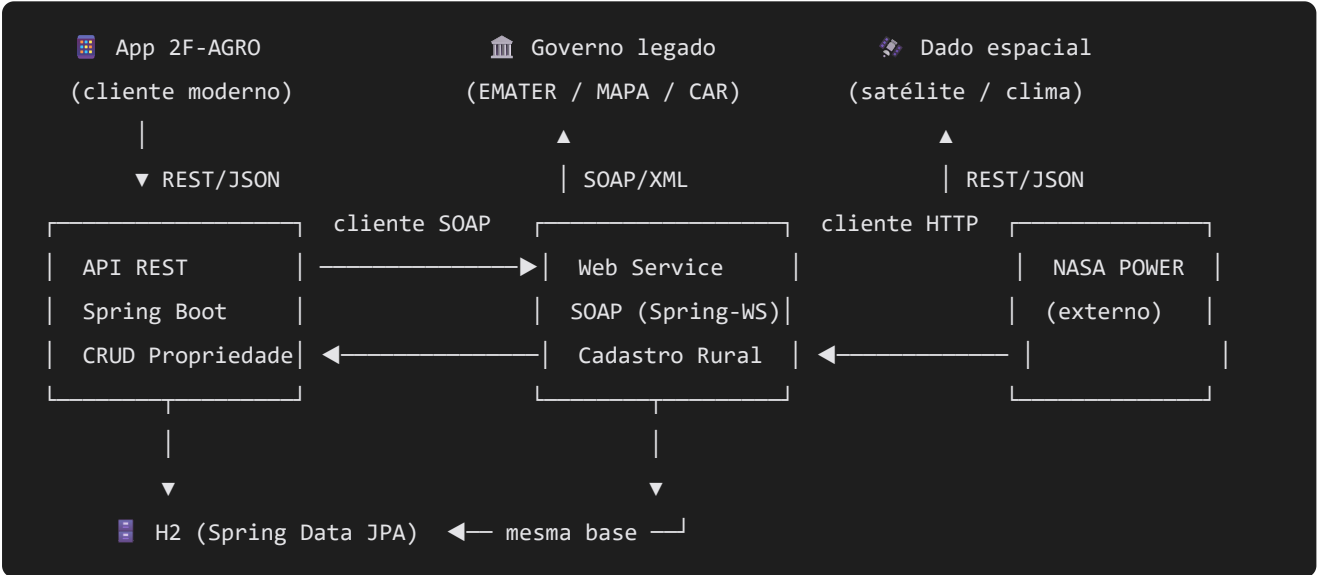
- Demonstrar os princípios **SOA**: baixo acoplamento, reutilização, contratos de serviço, interoperabilidade (REST ↔ SOAP) e separação de responsabilidades.
- Aplicar **POO** (abstração, encapsulamento, herança, polimorfismo) no domínio.
- Entregar persistência, validação, tratamento de erros e integração resiliente.

2. Arquitetura da solução

A solução adota uma **arquitetura em camadas orientada a serviços**. Na borda, dois serviços expõem o sistema: uma **API REST** (Spring MVC) que atende clientes modernos em JSON e um **Web Service SOAP** (Spring-WS, *contract-first*) que simula o sistema legado do governo e publica seu **WSDL**. Ambos compartilham a mesma camada de **domínio** e de **persistência** (Spring Data JPA + H2), o que garante reúso e evita duplicação de regras.

Uma camada de **serviço** concentra as regras de negócio e a **orquestração da integração**, que combina os dois serviços internos (REST ↔ SOAP) com um **serviço externo de dados espaciais** (NASA POWER). O acoplamento é mantido baixo por **interfaces** (ex.: `ServicoClimatico`) e por **DTOs**, que isolam o contrato público do modelo interno — permitindo, por exemplo, trocar a fonte climática (NASA → CPTEC) sem impacto no restante do sistema.

2.1 Diagrama de Arquitetura SOA



Camadas (pacote `br.com.fiap.agro.soa`): `rest` (controllers) · `soap` (endpoint + cliente) · `service` (regras + integração) · `repository` (JPA) · `domain` (POO) · `dto`.

2.2 Princípios SOA aplicados

Princípio	Como é atendido na solução
Baixo acoplamento	Interfaces (<code>ServicoClimatico</code>) e DTOs isolam contrato de implementação
Reutilização de serviços	Domínio e repositórios JPA reaproveitados por REST e SOAP
Interoperabilidade	REST/JSON ↔ SOAP/XML + consumo de API externa (NASA POWER)
Contratos de serviço	XSD/WSDL no SOAP e contrato REST documentado (endpoints/DTOs)

Princípio	Como é atendido na solução
Integração entre sistemas	Orquestrador que combina REST, SOAP e serviço externo
Separação de responsabilidades	Camadas <code>rest / soap / service / repository / domain / dto</code>

2.3 Estrutura do projeto

```

2f-agro-soa/
├── pom.xml # Maven (Java 17, Spring Boot 3.3.5)
├── README.md
├── docs/ # documentação e evidências
│   ├── DOCUMENTACAO-SOA.md / .pdf
│   ├── evidencias/EVIDENCIAS.md
│   ├── postman/2f-agro-soa.postman_collection.json
│   └── soapui/ (README + requests SOAP)
├── src/main/
│   ├── java/br/com/fiap/agro/soa/
│   │   ├── SoaApplication.java
│   │   ├── rest/ PropriedadeController · IntegracaoController · GlobalExceptionHandler
│   │   ├── soap/ WebServiceConfig · CadastroRuralEndpoint · CadastroRuralClient · SoapClient
│   │   ├── service/ PropriedadeService · IntegracaoService · ServicoClimatico · NasaPowerService
│   │   ├── repository/ PropriedadeRepository · CadastroRuralRepository (Spring Data JPA)
│   │   ├── domain/ Propriedade · CadastroRural · Alerta(+ Seca/Geada/Praga) · DadosClimatico
│   │   └── dto/ PropriedadeRequest/Response · CadastroIntegradoResponse · ErroResposta
│   └── resources/
│       ├── application.yml # configuração (H2, JPA, porta 8080)
│       ├── data.sql # seed inicial de propriedades
│       └── xsd/cadastro-rural.xsd # contrato SOAP (contract-first)

```

2.4 Persistência de dados (H2 + Spring Data JPA)

A aplicação persiste em um banco **H2 em memória** (`jdbc:h2:mem:agrodb`), recriado a cada inicialização. A camada de dados usa **Spring Data JPA**:

- **Entidades** `@Entity` mapeadas com `@Id` + `@GeneratedValue(IDENTITY)` : `Propriedade` (tabela `propriedade`) e `CadastroRural` (tabela `cadastro_rural`).
- **Repositórios** que estendem `JpaRepository`, ganhando o CRUD completo (`findAll`, `findById`, `save`, `deleteById`, `existsById`) sem implementação manual; o `CadastroRuralRepository` ainda deriva consultas por nome: `findByCpf`, `findByCar`.
- **Seed** (`data.sql`) carregado no startup (com `defer-datasource-initialization`), populando propriedades de exemplo para teste imediato.
- **Hibernate** com `ddl-auto: update` cria/atualiza o schema automaticamente.

H2 Console (inspeção do banco em runtime): `http://localhost:8080/h2-console` — JDBC URL `jdbc:h2:mem:agrodb`, usuário `sa`, sem senha.

3. API REST (CRUD de Propriedade)

Base: `http://localhost:8080/api/propriedades`

Método	Caminho	Ação	Status sucesso
GET	<code>/api/propriedades</code>	Lista todas	200
GET	<code>/api/propriedades/{id}</code>	Busca por id	200 (404 se não existe)
POST	<code>/api/propriedades</code>	Cria	201 (+ header <code>Location</code>)
PUT	<code>/api/propriedades/{id}</code>	Atualiza	200 (404 se não existe)
DELETE	<code>/api/propriedades/{id}</code>	Remove	204 (404 se não existe)

Validação (Bean Validation): `produtor/cultura/municipio` obrigatórios, `uf` com 2 letras, `latitude` ∈ [-90,90], `longitude` ∈ [-180,180], `areaHa` > 0. **Erros** centralizados (`@RestControllerAdvice`) com payload padronizado.

Exemplo — POST (201)

Request:

```
{ "produtor": "Joao Silva", "cultura": "Soja", "latitude": -23.5,
  "longitude": -46.6, "areaHa": 120.5, "municipio": "Ribeirao Preto", "uf": "SP" }
```

Response 201 :

```
{ "id": 1, "produtor": "Joao Silva", "cultura": "Soja", "latitude": -23.5,
  "longitude": -46.6, "areaHa": 120.5, "municipio": "Ribeirao Preto", "uf": "SP" }
```

Exemplo — validação (400)

Response 400 :

```
{ "status": 400, "erro": "Bad Request",
  "mensagem": "Falha de validação nos campos enviados",
  "caminho": "/api/propriedades",
  "campos": { "latitude": "latitude deve ser <= 90", "uf": "uf deve ter exatamente 2 letras",
    "produtor": "produtor é obrigatório", "areaHa": "areaHa deve ser positiva" } }
```

4. Web Service SOAP (Cadastro Rural)

Abordagem **contract-first**: o contrato `cadastro-rural.xsd` define os tipos; o Spring-WS publica o **WSDL** dinamicamente.

- **WSDL:** `http://localhost:8080/ws/cadastro-rural.wsdl`
- **Endpoint SOAP:** POST `http://localhost:8080/ws`
- **Namespace:** `http://fiap.com.br/agro/soa/cadastro-rural`

Operação	Entrada	Saída
<code>consultarCadastroRural</code>	<code>cpf</code> e/ou <code>car</code>	<code>encontrado</code> + bloco <code>cadastro</code>
<code>registrarCadastroRural</code>	dados do produtor	<code>protocolo</code> , <code>situacao</code> , <code>mensagem</code>

Exemplo — registrar

Request:

```
<cad:registrarCadastroRuralRequest xmlns:cad="http://fiap.com.br/agro/soa/cadastro-rural">
  <cad:cpf>123.456.789-00</cad:cpf>
  <cad:car>CAR-SP-001</cad:car>
  <cad:nomeProdutor>Joao Silva</cad:nomeProdutor>
  <cad:municipio>Ribeirao Preto</cad:municipio>
  <cad:uf>SP</cad:uf>
  <cad:areaHa>120.5</cad:areaHa>
</cad:registrarCadastroRuralRequest>
```

Response:

```
<ns2:registrarCadastroRuralResponse xmlns:ns2="http://fiap.com.br/agro/soa/cadastro-rural">
  <ns2:protocolo>CAR-1</ns2:protocolo>
  <ns2:situacao>REGISTRADO</ns2:situacao>
  <ns2:mensagem>Cadastro rural registrado com sucesso para Joao Silva</ns2:mensagem>
</ns2:registrarCadastroRuralResponse>
```

5. Integração entre serviços

Endpoint orquestrador: POST `/api/integracao/propriedades`

POST /api/integracao/propriedades

- └(1) cria a propriedade REST → H2 (Spring Data JPA)
- └(2) registra no "governo" SOAP → WebServiceTemplate (cliente JAXB) → /ws
- └(3) consulta clima do ponto REST → NASA POWER (RestClient, timeout+fallback)
- └(4) deriva alerta agroclimático .. POO → Alerta.gerarMensagem() (polimorfismo)

- **Serviço externo (especial):** `NasaPowerServicoClimatico` consome a API *climatology* da NASA POWER via `RestClient` (timeouts 5s/8s) e implementa a abstração `ServicoClimatico`.
- **REST ↔ SOAP interno:** `CadastroRuralClient` (`WebServiceTemplate` + JAXB) chama o próprio Web Service SOAP pela rede.
- **Resiliência:** falha de qualquer serviço externo **não derruba** o fluxo — vira `aviso` no resultado (`climaDisponivel=false` / `protocoloGoverno=null`), e a propriedade é criada.

Exemplo — enriquecida (clima real NASA)

```
{ "propriedade": { "id": 1, "municipio": "Ribeirão Preto", "uf": "SP" },
  "climaDisponivel": true,
  "clima": { "temperaturaMediaC": 23.45, "precipitacaoMm": 3.48, "umidadeRelativa": 68.97, "fonte": "NASA",
  "alerta": null }
```

6. Princípios de POO aplicados

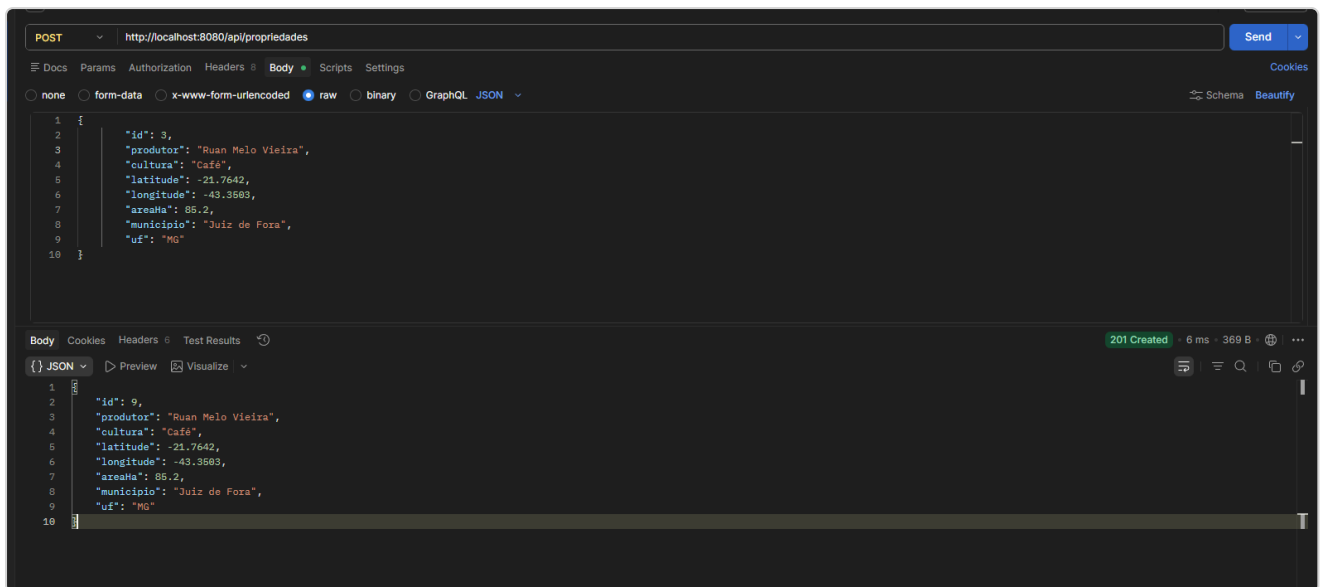
Pilar	Onde
Abstração	<code>interface ServicoClimatico</code> (NASA/CPTEC intercambiáveis)
Encapsulamento	entidades com campos privados + validação nos setters/construtor
Herança	<code>abstract Alerta</code> → <code>AlertaSeca</code> , <code>AlertaGeada</code> , <code>AlertaPraga</code>
Polimorfismo	<code>alerta.gerarMensagem()</code> sobrescrito por subtipo

7. Evidências de funcionamento

Request/response **reais** capturados com a aplicação rodando, via Postman. Coleções no repositório: [Postman](#) e [SoapUI](#).

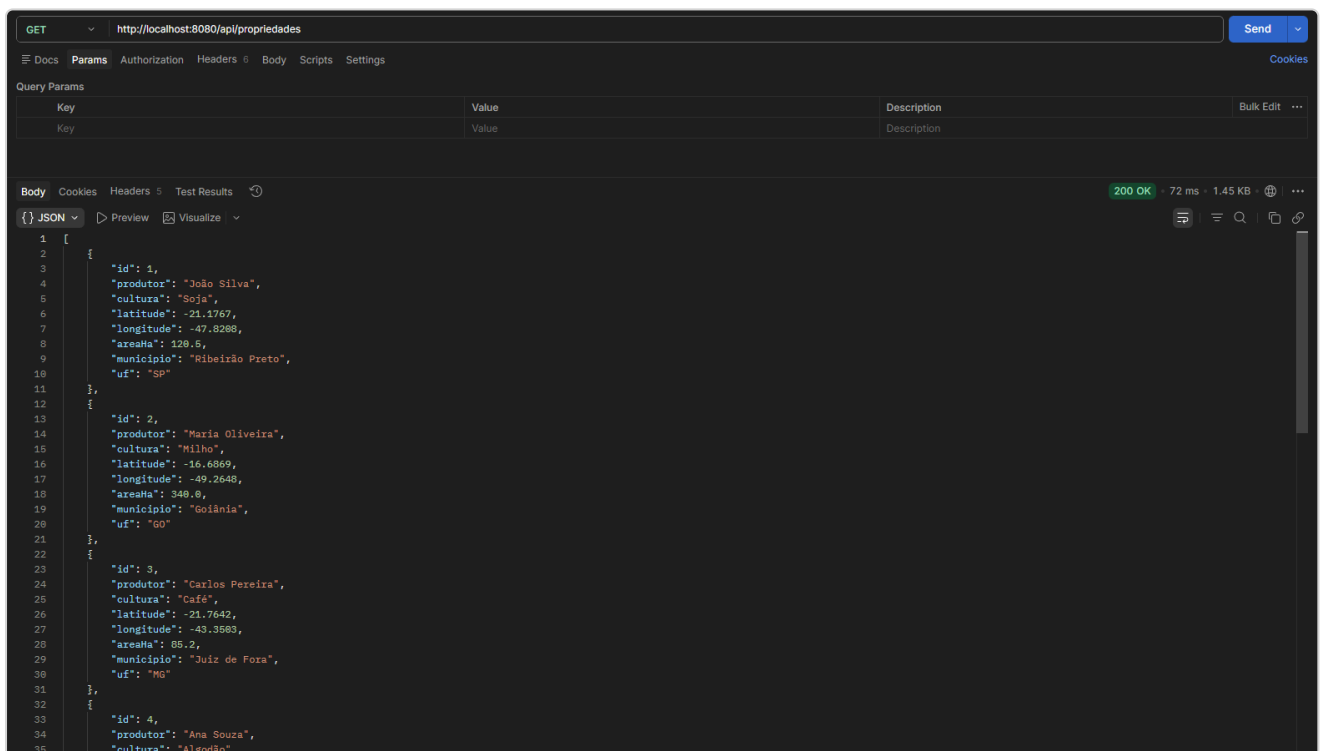
7.1 REST — criar propriedade (201 Created)

POST /api/propriedades → **201** retornando o objeto com o `id` gerado.



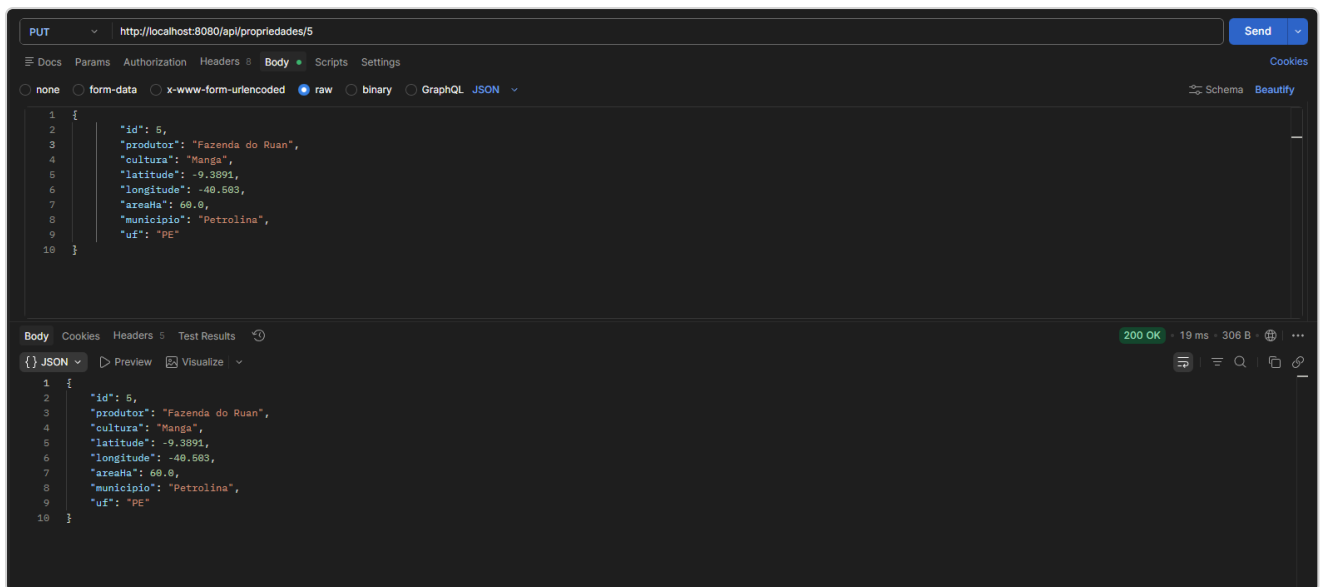
7.2 REST — listar (200 OK)

GET /api/propriedades → 200 com a lista (seed + criadas).



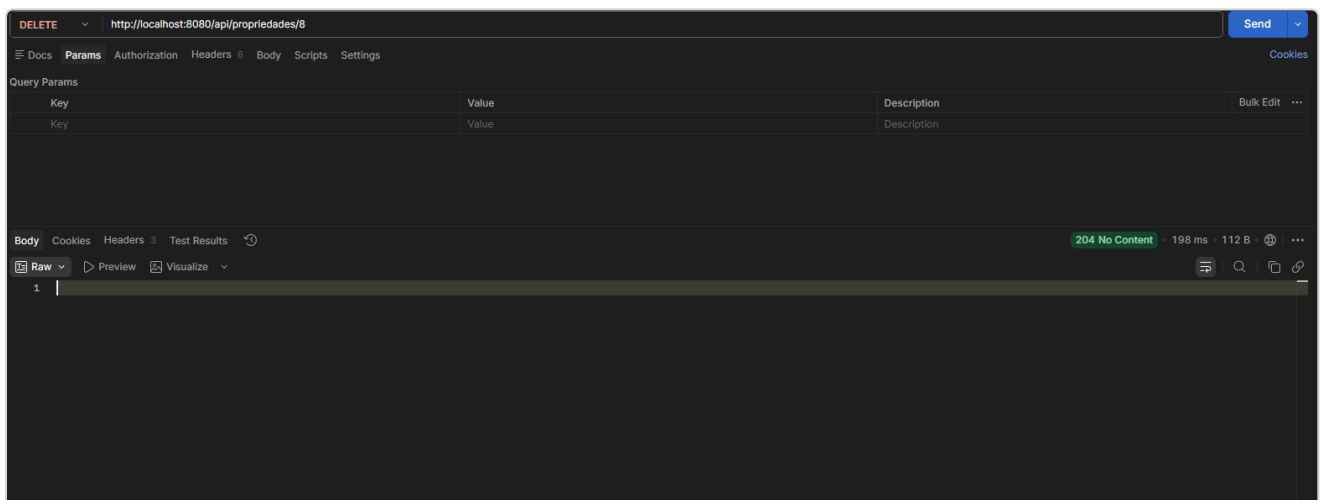
7.3 REST — atualizar (200 OK)

PUT /api/propriedades/5 → 200 com o objeto atualizado.



7.4 REST — remover (204 No Content)

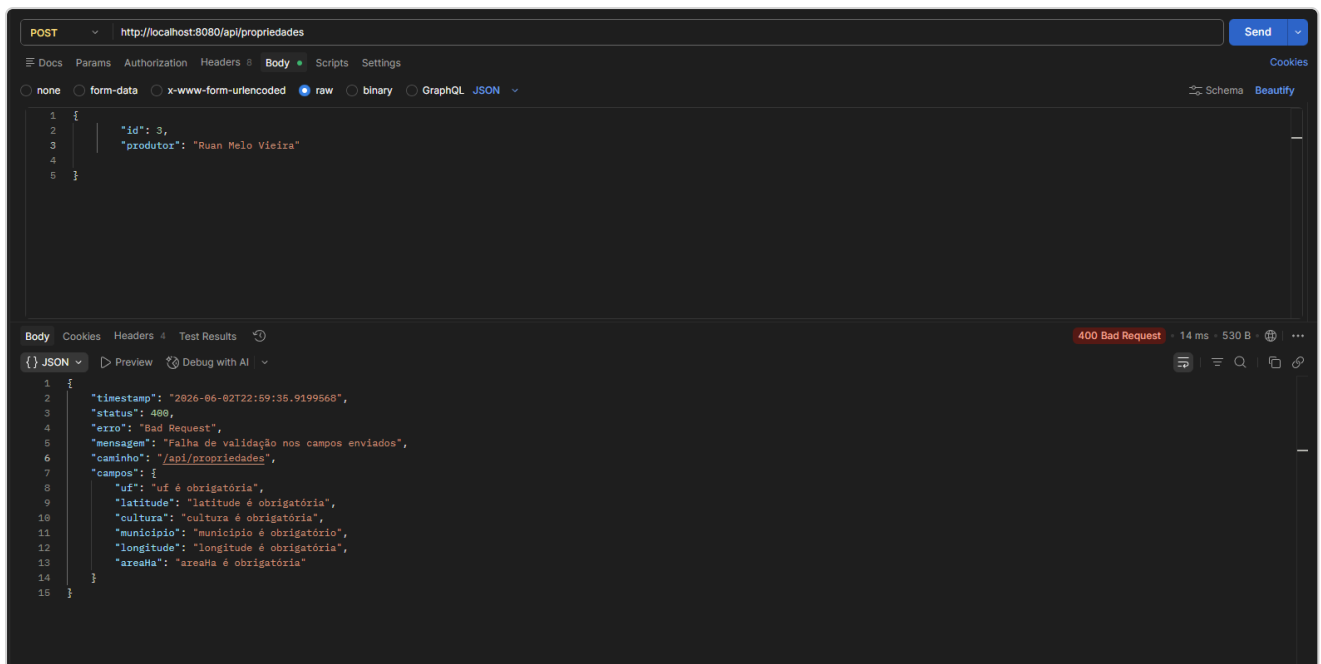
DELETE /api/propriedades/8 → **204** sem corpo.



7.5 REST — validação (400 Bad Request)

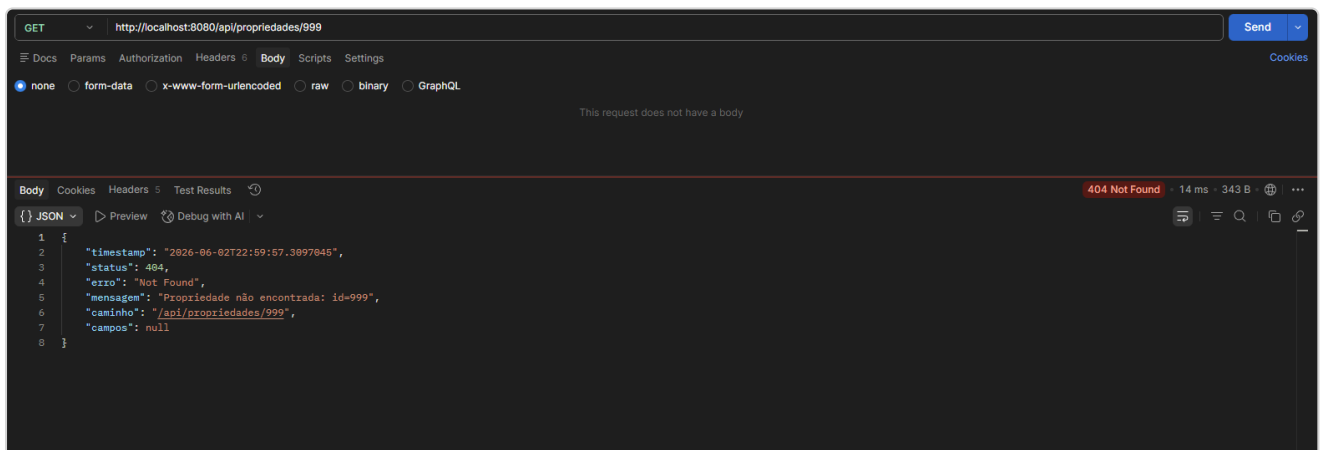
POST com campos faltando → **400** com detalhamento campo a campo:

```
{ "status": 400, "erro": "Bad Request",
  "mensagem": "Falha de validação nos campos enviados",
  "caminho": "/api/propriedades",
  "campos": { "uf": "uf é obrigatória", "latitude": "latitude é obrigatória",
    "cultura": "cultura é obrigatória", "municipio": "município é obrigatório",
    "longitude": "longitude é obrigatória", "areaHa": "areaHa é obrigatória" } }
```

7.6 REST — não encontrado (404 Not Found)

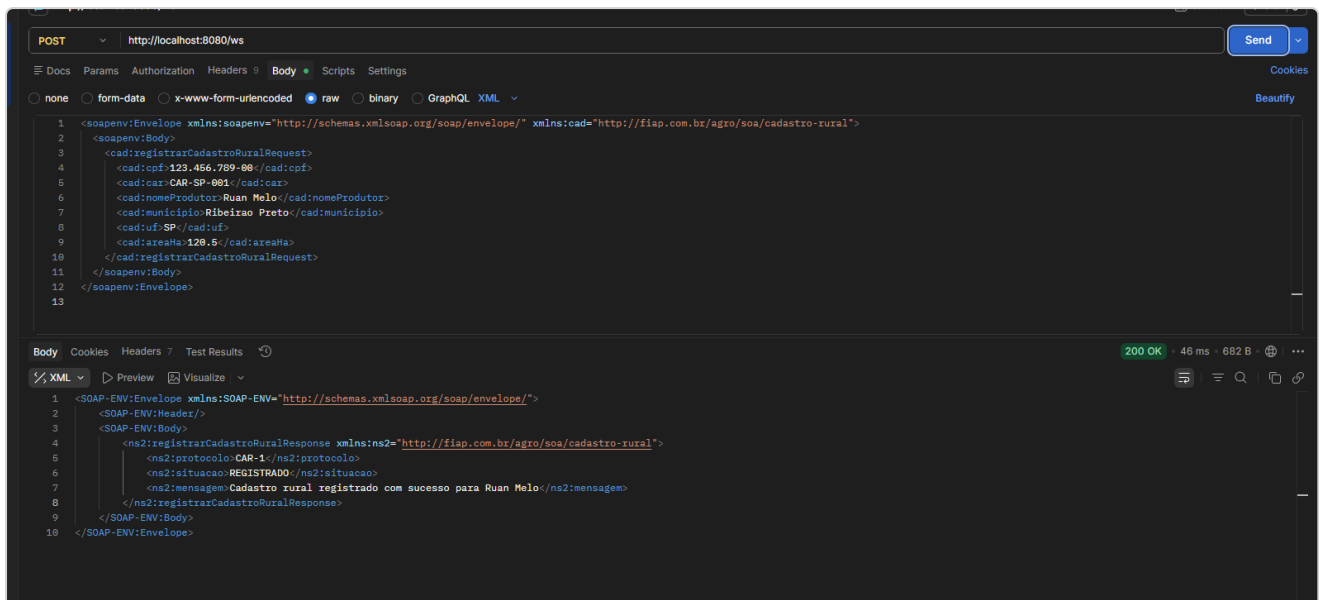
GET /api/propriedades/999 → **404** com mensagem padronizada.



7.7 SOAP — registrar cadastro

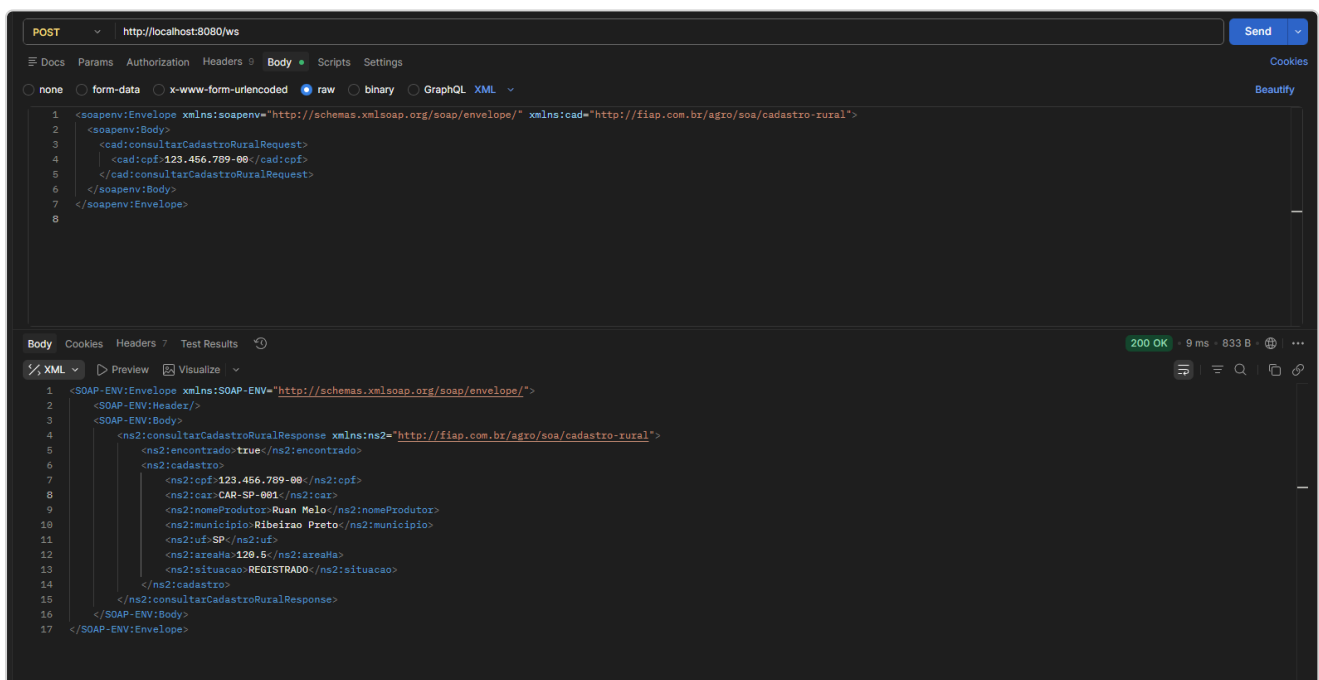
registrarCadastroRural → resposta com protocolo:

```
<ns2:registrarCadastroRuralResponse xmlns:ns2="http://fiap.com.br/agro/soa/cadastro-rural">
  <ns2:protocolo>CAR-1</ns2:protocolo>
  <ns2:situacao>REGISTRADO</ns2:situacao>
  <ns2:mensagem>Cadastro rural registrado com sucesso para Joao Silva</ns2:mensagem>
</ns2:registrarCadastroRuralResponse>
```



7.8 SOAP — consultar cadastro

consultarCadastrorRural (encontrado) → <encontrado>true</encontrado> + bloco <cadastro> ; (não encontrado) → <encontrado>false</encontrado> .



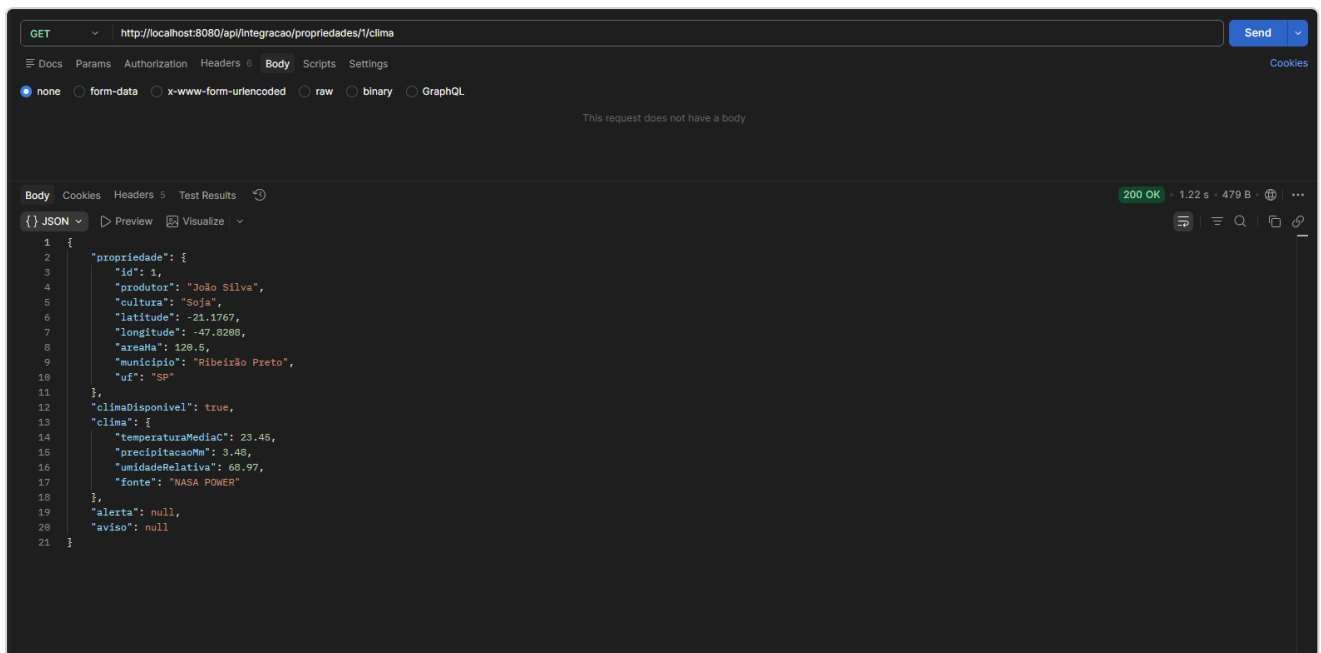
7.9 Integração — enriquecimento com clima real da NASA POWER (200)

GET /api/integracao/propriedades/1/clima (Ribeirão Preto/SP):

```

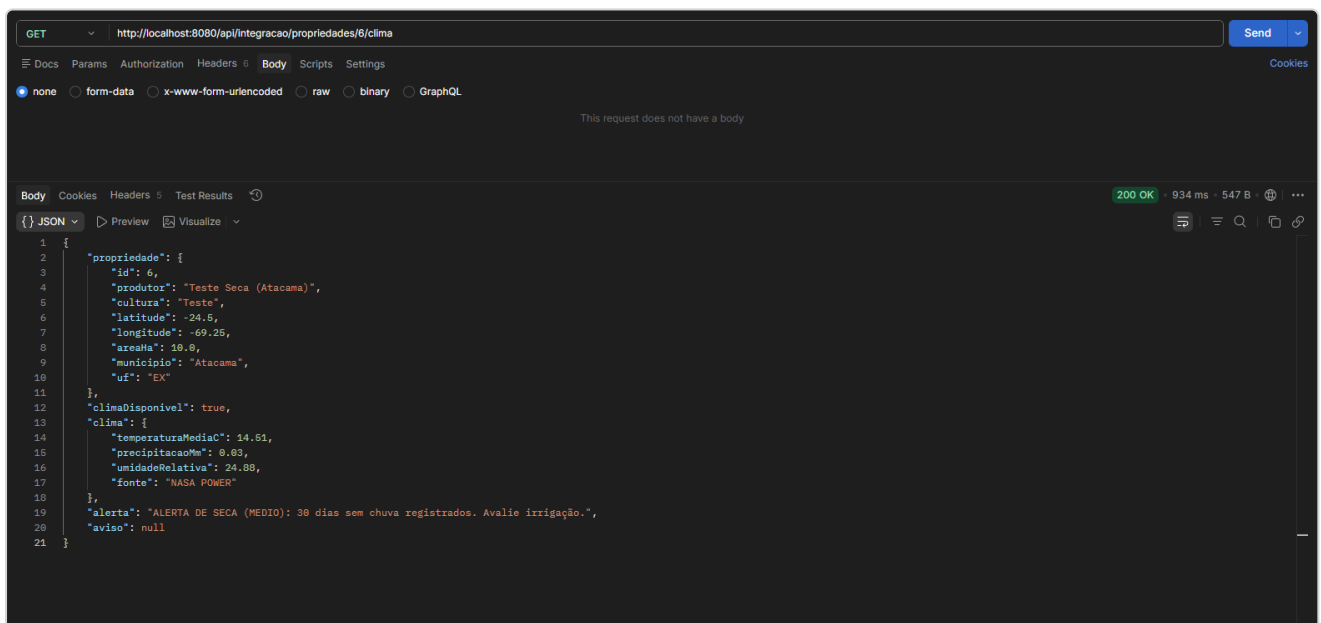
{ "climaDisponivel": true,
  "clima": { "temperaturaMediaC": 23.45, "precipitacaoMm": 3.48, "umidadeRelativa": 68.97, "fonte": "NASA POWER",
  "alerta": null }
}

```



7.10 Integração — alerta agroclimático (polimorfismo)

GET /api/integracao/propriedades/6/clima (Atacama, precipitação 0,03 mm/dia) dispara o alerta: "alerta": "ALERTA DE SECA (MEDIO): 30 dias sem chuva registrados. Avalie irrigação."



7.11 Integração — fallback resiliente (serviços externos fora do ar)

O fluxo **não quebra**: a propriedade é criada e as falhas viram avisos.

```
{ "propriedade": { "id": 5, "produtor": "Teste Resiliencia" },
  "protocoloGoverno": null, "climaDisponivel": false, "clima": null,
  "avisos": [ "Registro no governo (SOAP) indisponível: Connection refused",
    "Dado climático indisponível (NASA POWER): ..." ] }
```

8. Tecnologias utilizadas

- **Java 17, Spring Boot 3.3.5**
 - **Spring Web** (REST), **Spring Web Services / Spring-WS** (SOAP contract-first), **WSDL4J**
 - **Spring Data JPA + H2** (banco em memória)
 - **Bean Validation** (Hibernate Validator)
 - **JAXB** (geração a partir do XSD), **RestClient** (cliente HTTP), **WebServiceTemplate** (cliente SOAP)
 - **Maven**
-

9. ODS relacionados

A solução se relaciona aos Objetivos de Desenvolvimento Sustentável:

- **ODS 1 – Fome Zero e Agricultura Sustentável:** monitoramento agrícola com dados de satélite apoia a produção de alimentos e o manejo sustentável da propriedade rural.
 - **ODS 5 – Ação Contra a Mudança Global do Clima:** uso de dados climáticos (NASA POWER) e geração de alertas de seca/geada ajudam na adaptação e prevenção de perdas no campo.
 - **ODS 3 – Indústria, Inovação e Infraestrutura:** integração de sistemas distribuídos (REST + SOAP + dados espaciais) como infraestrutura tecnológica inovadora.
-

10. Conclusão

O projeto demonstra, de ponta a ponta, os princípios de **SOA**: serviços com contratos bem definidos (REST e WSDL/SOAP), **interoperabilidade** entre um cliente moderno e um sistema legado, **integração** com um serviço externo real (NASA POWER) e **baixo acoplamento** via interfaces e DTOs. A arquitetura em camadas e a aplicação dos pilares de **POO** tornam o código coeso, testável e extensível (ex.: trocar a fonte climática para CPTEC sem alterar o restante). O tratamento resiliente de falhas garante disponibilidade do fluxo principal mesmo quando serviços externos ficam indisponíveis.